



www.devmedia.com.br

[versão para impressão]

Link original: <http://www.devmedia.com.br/articles/viewcomp.asp?comp=30059>

Modelos de Processos Ágeis: conceitos e princípios

Veja neste artigo uma introdução aos modelos de processos ágeis. Estudaremos os principais conceitos e princípios dos modelos de processo ASD, Scrum, DSDM, FDD, LSD, AM e AUP.

Nas últimas décadas temos visto o surgimento de diversos modelos de processos ágeis. Alguns modelos tem certas características que os diferenciam dos demais. Essas características serão vistas neste artigo. Cada modelo de processos possui suas características, mas é importante saber que todos os modelos estão em conformidade com o Manifesto Ágil. Nas próximas seções analisaremos melhor cada um desses modelos de processos utilizados em diversas organizações e muito comentado na bibliografia atualmente. Esses modelos de processos são mundialmente utilizados e podem dar boas dicas ao leitor sobre quais modelos podem ser mais adequados ao seu contexto. É

sempre importante ressaltar que muitas vezes o melhor processo para o nosso ambiente empresarial é uma mescla de vários modelos de processo ou então um processo específico. Devemos sempre analisar as nossas necessidades e verificar quais modelos melhor se adequam ao nosso contexto.

Desenvolvimento de Software Adaptativo (ASD)

Jim Highsmith propôs o Desenvolvimento de Software Adaptativo (Adaptative Software Development - ASD) como uma técnica para construção de software e sistemas altamente complexos. Esse modelo se concentra na colaboração e auto-organização das equipes.

O criador do modelo Adaptativo define um ciclo de vida para o modelo baseando-se em três fases: especulação, colaboração e aprendizagem.

- Na fase de especulação o projeto é iniciado e tem-se o planejamento de ciclos adaptáveis. Esse planejamento de ciclos adaptáveis usa as informações contidas no início do projeto como: a missão do cliente, restrições do projeto e os requisitos básicos. Os requisitos básicos serão utilizados para definir o conjunto de ciclos da versão, ou seja, os incrementos de software operacional. Vale salientar que esse plano de ciclos sofrerá mudanças. Após completar cada ciclo o plano é revisto e ajustado para que tenhamos o trabalho reajustado à realidade que a equipe ASD está trabalhando.
- A colaboração é um tema bastante discutido e enfatizado nos métodos ágeis. A colaboração envolve confiança, críticas sem animosidade, auxílio, trabalho árduo, comunicação dos problemas ou preocupações de forma a conduzir ações efetivas, etc. Dessa

forma, a colaboração ajuda bastante no levantamento de necessidades, especificações, etc.

- O ASD também enfatiza que o aprendizado é um elemento-chave para que possamos conseguir uma equipe auto-organizada. O criador do método Highsmith argumenta que os desenvolvedores superestimam o seu próprio entendimento quanto à tecnologia, processo ou mesmo quanto ao projeto. Dessa forma, Highsmith enfatiza que o aprendizado irá ajudar a todos os desenvolvedores a aumentar os níveis reais de entendimento. Com base nisso, as equipes ASD aprendem através de três maneiras: grupos focados, revisões técnicas e autópsias de projetos.

SCRUM

O Scrum tem este nome devido a uma atividade que ocorre durante uma partida de rugby, esporte muito popular nos Estados Unidos e evoluindo bastante no Brasil com times profissionais surgindo a cada dia. Scrum é um método de desenvolvimento ágil de software criado por Jeff Sutherland e a sua equipe de desenvolvimento de software. O método foi criado no início dos anos 90 e recentemente ganhou incrementos nos métodos gráficos através de Schwaber e Beedle. Assim como os demais modelos de desenvolvimento ágil o Scrum possui princípios consistentes com o manifesto ágil. O Scrum é indicado para projetos com prazos apertados, requisitos que estão sempre sendo modificados e que são críticos para o negócio. Este método define um conjunto de ações de desenvolvimento, são eles: o Backlog que é onde registra-se todo o trabalho pendente (requisitos ou funcionalidades) organizando-os por prioridades. Ressalta-se que novas funcionalidades podem ser adicionadas a esse Backlog a qualquer momento introduzindo as alterações do usuário. Porém, o gerente do produto deve avaliar esta funcionalidade e atualizar as prioridades; Temos também como uma ação do Scrum as Sprints que são algumas

funcionalidades do Backlog que podem ser atendidas num prazo curto, de no máximo 30 dias. É nas Sprints que o trabalho de desenvolvimento realmente será realizado para entregar o mais rápido possível um incremento de software operacional ao cliente. Quando as Sprints já estão ocorrendo não devemos fazer alterações e possíveis modificações devem ser realizadas nas próximas Sprints; Outra ação do Scrum são as Reuniões que tipicamente são curtas (15 minutos) e realizadas diariamente pela equipe Scrum. Nesta reunião diária são feitas três perguntas bastante pontuais para todos os membros da equipe: "O que você realizou desde a última reunião da equipe?", "Quais obstáculos estão encontrando?" e "O que planeja realizar até a próxima reunião da equipe?". Estas perguntas ajudam todos a entenderem o que cada um fez no dia anterior, se tem alguma dificuldade para realizar o trabalho atual e o que se pretende fazer hoje. Esta reunião é considerada muito importante porque ajuda a equipe a revelar problemas o mais cedo possível, também ajuda a disseminar o conhecimento. Uma dica importante é que nem todas as equipes acham a reunião diária apropriada para ser feita na parte da manhã, isto porque alguns membros começam a trabalhar no turno da tarde ou porque as pessoas não possuem um humor muito bom pela manhã, tente avaliar essas questões e decida o melhor momento.

No Scrum também temos um líder de equipe que é chamado de Scrum Master. Ele é responsável por conduzir a reunião diária e avaliar as respostas dos integrantes. O objetivo do Scrum Master também é remover obstáculos.

Por fim, o SCRUM também trabalha com a ideia de entrega de incrementos de software ao cliente, ou Demos. Essa funcionalidade permite ao cliente avaliar e dar feedbacks para a equipe. Como as Demos são realizadas durante os incrementos, pode ser que não tenhamos toda a funcionalidade completa, mas teremos funções que possam ser entregues dentro do prazo acertado com o cliente.

Método de desenvolvimento de sistemas dinâmicos (DSDM)

O Dynamic System Development Method (Método de Desenvolvimento de Sistemas Dinâmicos) ou DSDM é mais uma abordagem de desenvolvimento de software ágil que oferece uma metodologia para construir e manter sistemas que atendem restrições de prazo apertado através do uso da prototipagem incremental. Este método é mantido pela DSDM Consortium (www.dsdm.org). Este grupo é mundial e conta com diversas empresas que são membros do grupo que mantém o DSDM.

O DSDM é um processo iterativo que a cada iteração (ou incremento) possui somente a quantidade de trabalho suficiente. Isto facilita o movimento para o próximo incremento. Detalhes remanescentes são completados depois quando outros requisitos forem conhecidos ou alterações tiverem sido solicitadas pelo cliente.

No DSDM temos a definição de um modelo de processos ágeis, chamado ciclo de vida DSDM, que define três ciclos iterativos diferentes que são precedidos por duas atividades de ciclo de vida adicionais:

- O Estudo da Viabilidade, que estabelece requisitos básicos de negócio e restrições associados à aplicação a ser construída e depois avalia se a aplicação é ou não um candidato viável para o processo DSDM;
- O Estudo do Negócio que é responsável por estabelecer os requisitos funcionais e de informação que permitirão à aplicação agregar valor de negócio, e também temos a definição básica da arquitetura da aplicação e a identificação dos requisitos de facilidade de manutenção para a aplicação;

Os três ciclos iterativos são:

- A Iteração de Modelos Funcionais onde é produzido um conjunto de protótipos incrementais que demonstram a funcionalidade para o cliente. Neste ciclo iterativo procura-se juntar os requisitos adicionais ao se obter feedback dos usuários conforme eles vão testando e utilizando o protótipo;
- A Iteração de Projeto e Desenvolvimento onde revisitamos os protótipos desenvolvidos durante a iteração de modelos funcionais para que possamos assegurar que cada um tenha passado por um processo de engenharia para capacitá-los a oferecer aos usuários finais valor de negócio em termos operacionais;
- Por fim temos a Implementação que é onde de fato colocamos a última versão do incremento de software (um protótipo operacionalizado) no ambiente operacional. Nesta última fase o incremento pode não estar 100% completo ou ainda alterações podem ser solicitadas conforme o incremento seja alocado. Em qualquer dos casos, o trabalho do desenvolvimento do DSDM continua, retornando-se à atividade de iteração do modelo funcional.

Desenvolvimento dirigido à funcionalidade (FDD)

Peter Coad e seus colegas criaram um modelo de processo prático para a engenharia de software orientada a objetos chamada Desenvolvimento Dirigido a Funcionalidade ou Feature Driven Development (FDD). Após o trabalho de Coad surgiu outro modelo de processo que aperfeiçoou o trabalho desenvolvido. Este trabalho foi realizado por Stephen Palmer e John Felsing que descreveram um processo ágil adaptativo que pode ser aplicado em projetos de médio e grande porte.

Seguindo as outras abordagens ágeis, o FDD também evidencia a colaboração entre as pessoas da equipe, gerencia os problemas e complexidades de projetos utilizando a decomposição baseada em funcionalidades que é seguida pela integração dos incrementos de software e por fim também enfatiza a comunicação de detalhes técnicos usando formas verbais, gráficos e texto. Além disso, o FDD enfatiza as atividades de garantia da qualidade por meio da estratégia de desenvolvimento incremental, inspeções de código e de projeto, auditorias, coleta de métricas e o uso de padrões para análise, projeto e construção.

O FDD preconiza que a funcionalidade é uma função valorizada pelo cliente passível de ser implementada em até duas semanas ou menos. Essa ênfase na definição de funcionalidades tem como benefícios uma descrição mais fácil das funcionalidades por parte do usuário, uma melhor compreensão como elas se relacionam entre si e uma melhor revisão das ambiguidades, erros ou omissões. Isso ocorre porque elas são pequenas, formadas em pequenos blocos. Além disso, outro benefício é que as funcionalidades podem ser organizadas em um agrupamento hierárquico relacionados com o negócio, pode ser entregue a cada duas semanas, o projeto e o código são mais facilmente inspecionados e o planejamento, cronograma e acompanhamento do projeto são guiados pela hierarquia de funcionalidades ao invés de tarefas de engenharia de software.

Coad e seus colegas sugerem os seguintes modelos para definirmos uma funcionalidade:

<ação> o <resultado> <por|para quem|de|para que> um <objeto>

Onde objeto é "uma pessoa, local ou coisa".

Um exemplo de funcionalidades poderia ser:

Adicione o produto ao carrinho

Mostre as especificações técnicas do produto

O FDD também define cinco processos, são eles: Desenvolver um Modelo Geral, Construir uma Lista de Funcionalidades, Planejar por Funcionalidades, Projetar por Funcionalidade, Desenvolver por Funcionalidade.

Um aspecto interessante do FDD em relação a outros métodos ágeis é que ele dá maior ênfase às diretrizes e técnicas de gerenciamento de projetos do que outros métodos ágeis disponíveis.

Desenvolvimento de software Enxuto (LSD)

O Desenvolvimento de Software Enxuto ou Lean Software Development (LSD) adaptou os princípios da fabricação enxuta da indústria para o mundo da engenharia de software. Entre os princípios do desenvolvimento enxuto tem-se: eliminar desperdícios, incorporar qualidade, criar conhecimento, adiar compromissos, entrega rápida, respeitar as pessoas e otimizar o todo.

Cada um desses princípios foi adaptado ao contexto do processo de software. Para citar um exemplo, o eliminar desperdício, no contexto de um projeto de software ágil é interpretado como não adicionar recursos ou funções obscuras, avaliar possíveis impactos do custo e cronograma de qualquer requisito que seja solicitado, eliminar etapas do processo que sejam supérfluos, etc.

Modelagem Ágil (AM)

Muitas vezes os engenheiros de software precisam desenvolver sistemas grandes e complexos. O escopo e a complexidade desses sistemas são modelados de forma para que todos os envolvidos no projeto entendam quais requisitos devem ser integrados, para que os problemas possam ser melhores subdivididos entre as pessoas que têm de solucioná-los e para que a qualidade possa ser avaliada enquanto estamos projetando e desenvolvendo o sistema.

Muitas notações e métodos de modelos têm surgidos ao longo das últimas décadas. Esses métodos e notações auxiliam tanto a análise quanto o projeto e muitas vezes geram implementações baseando-se nesses modelos. Apesar de eles terem evoluído, ainda temos o problema de mantê-los. A cada mudança devemos atualizar os modelos e muitas vezes cuidando com o grau de formalismo que é imposto.

Os métodos ágeis oferecem uma alternativa para a modelagem de engenharia de software. Scott Ambler descreve o que ele chama de modelagem ágil da seguinte forma: "Modelagem ágil (AM) consiste em uma metodologia baseada na prática, voltada para a modelagem e documentação de sistemas com base em software. Simplificando, modelagem ágil consiste em um conjunto de valores, princípios e práticas voltadas para a modelagem do software que podem ser aplicados em um projeto de desenvolvimento de software de forma leve e efetiva. Os modelos ágeis são mais efetivos que os tradicionais pelo fato de serem meramente bons, pois não têm a obrigação de ser perfeitos".

Todos os valores do manifesto ágil são consistentes com o que é pregado na modelagem ágil. Entre as principais características da modelagem ágil tem-se:

- Modelar com um objetivo: O desenvolvedor sempre deverá ter um objetivo antes de criar um modelo como, por exemplo, comunicar informações ao cliente ou ajudar a compreender melhor algum aspecto que não esteja tão claro. Podemos notar que aqui a

modelagem ágil foca em pensarmos antes de criarmos modelos simplesmente por criar ou para guardar modelos. O modelo criado deverá realmente servir para algum fim específico.

- Usar modelos múltiplos: Existem diversos modelos e notações que podemos utilizar para descrever o software, no entanto, apenas um subconjunto é realmente essencial para a maioria dos projetos. A modelagem ágil sugere que sejam utilizados os modelos que realmente comuniquem adequadamente para os usuários específicos. Além disso, sugere-se que devemos usar diferentes modelos para apresentar um aspecto diferente do sistema.
- Viajar leve: Devemos apenas conservar aqueles modelos que terão valor em longo prazo, os demais devem ser descartados. Isso ocorre porque efetuar manutenção em modelos é árduo, assim como qualquer outro artefato de software. Modelos são apenas mais um artefato para darmos manutenção.
- Conteúdo é mais importante do que representação: A modelagem deve sempre transmitir informação, seja para um cliente tentando entender alguma parte do sistema, seja o próprio engenheiro de software tentando tirar informações do cliente ou uma equipe de desenvolvimento de software tentar entender ou repassar algo. Dessa forma, um modelo sintaticamente perfeito que não transmite muito conteúdo não possui tanto valor quanto um modelo com notações falhas que fornece conteúdo valioso para seu público-alvo.
- Ter conhecimento e domínio dos modelos e ferramentas que serão utilizadas: Devemos compreender os pontos fortes e fracos de cada modelo e ferramenta usada para criá-lo. Isso dá mais visibilidade e possibilidade de resolver problemas da melhor forma.
- Adaptar localmente: Como sempre pregamos na agilidade, devemos adaptar a modelagem às necessidades da nossa equipe. Um exemplo disso é uma equipe que odiava mexer com as ferramentas de modelagem. Dessa forma, adaptamos a

modelagem para ser realizada num quadro, numa sala de reunião. A modelagem flui muito mais e a equipe pode se sentir mais animada.

Processo Unificado Ágil (AUP)

O Processo Unificado Ágil ou Agile Unified Process (AUP) adota uma filosofia serial (sequência linear de atividades) para o que é amplo e iterativo para o que é particular. Este processo possui atividades nas fases clássicas adotadas pelo Processo Unificado: Início, Elaboração, Construção e Transição. Dentro de cada uma das atividades a equipe itera ou se repete para alcançar a agilidade e entregar incrementos de software para os usuários finais. É importante que essa entrega seja mais rápida quanto possível.

A AUP possui seis atividades, na qual a equipe irá sempre iterar. As atividades são:

- Modelagem: Modelos UML representando o negócio são criados. Os modelos devem ser suficientemente bons e adequados para permanecer ágil.
- Implementação: Modelos são traduzidos para o código-fonte.
- Teste: A equipe projeta e executa uma série de teste a fim de descobrir possíveis erros e para assegurar que o código-fonte se ajuste aos requisitos.
- Aplicação: É a entrega de um incremento de software e a aquisição de feedback dos usuários finais com base neste incremento.
- Gerenciamento de Configuração e Gerenciamento de projeto: No contexto da AUP, gerenciamento de configuração refere-se ao gerenciamento de alterações, riscos e de controle de qualquer artefato persistente que são produzidos pela equipe. O

gerenciamento de projeto controla o progresso de uma equipe e coordena suas atividades.

- Gerenciamento de Ambiente: Coordena toda a infraestrutura de processos incluindo padrões, ferramenta e outras tecnologias de suporte disponíveis para a equipe.

Como vimos nesse artigo, um processo de software tem como característica um conjunto de atividade que conduzem o desenvolvimento do produto de software. Um processo tem como característica a definição de quem realiza uma atividade e quando fazê-la. Um Modelo de processo é uma descrição simplificada de um processo onde se definem as atividades, especificam-se os produtos gerados nas atividades e indicam-se os papéis das pessoas envolvidas. As empresas normalmente definem seus próprios modelos de processos, juntando o que tem de melhor nos diferentes modelos de processos existentes. Outras empresas preferem adotar um modelo de processo que seja mais adequado ao seu contexto.

Bibliografia

[1] Pressman, R. Engenharia de Software: Uma abordagem Profissional. 7º edição. Editora Bookman.

[2] Ken Schwaber e Jeff Sutherland. Scrum Guide. Disponível em <http://www.scrum.org>

[3] Mike Cohns: Succeeding with Agile. Disponível em <http://www.mountangoatsoftware.com/blog/>



por Higor Medeiros

Engenharia de software lover ❤
